

计算机图形学期末复习



风倾

电子科技大学 软件工程硕士在读

[收录于 · 课程复习](#)

86 人赞同了该文章

计算机图形学理论课复习大纲

题型：简答题，计算题，绘图题，分析与设计题，程序实现题。

目录

分数分别为：20分，30分，10分，30分，10分。

一、图形学基本概念

1. 什么是计算机图形学？
2. 计算机图形学的主要研究内容包括哪些？
3. 计算机图形学的主要应用领域。
4. 图形和图像的主要区别是什么？
5. 计算机图形处理系统的主要构成。
6. 显卡的工作流程。
7. 什么是显示存储器？
8. GPU的基本概念。
9. 显卡分辨率的含义。
10. GPU堆包含哪几种主要的类型，具有什么样的特点？
11. GPU渲染流水线包含哪几个阶段，分别完成什么任务？
12. 光照的基本原理
13. 光照模型的分类
14. 光源的分类
15. 纹理的表现形式。
16. 纹理的过滤方式及寻址方式
17. 多级纹理的基本原理
18. 模型的表示方法
19. Direct3D 12图形绘制的基本原理
20. 三维模型顶点的数据结构表示

二、图形学基本算法

要求：算法的基本原理，算法描述，算法执行过程分析，过程计算，编程实现等。

1. 直线生成算法：DDA算法，中点画线法、Bresenham法。
2. 圆和圆弧的生成算法：DDA算法，中点画线法，Bresenham法
3. 线宽生成算法：刷子绘制法，实区域填充法
4. 线型的处理
5. 区域填充法：扫描线法，边填充法，种子填充法，扫描线种子填充法，
6. 反走样技术：像素细分技术，Bresenham反走样技术
7. 曲线生成：Bezier曲线和B样条曲线，包括二阶、三阶曲线等。
8. 图形变化：平移、缩放、旋转
9. 坐标系之间的变换：局部坐标、世界坐标、摄像机坐标、屏幕坐标
10. 四边形裁剪算法：Cohen-Sutherland裁剪算法、中点分割裁剪算法、Liang-Barskey裁剪算法
11. 多边形裁剪算法：Sutherland-Hodgman算法，Weiler-Atherton算法
12. 三维线段裁剪算法：长方体裁剪算法，视锥体裁剪算法



14. 曲面的生成·Bezier 曲面的生成, B 样条曲面的生成, 特别是一阶和二阶曲面

15. 地形的生成算法

16. 雨雪的模拟算法

答案见:



风倾: 计算机图形学复习大纲总结

67 赞同 · 5 评论 文章

第1章 绪论

什么是计算机图形学?

计算机图形学是研究通过计算机将数据转化为图形, 并在专门显示设备上显示的原理、方法和技术的学科。也就是如何用计算机生成、处理和显示图形的一门学科。

计算机图形学是研究怎样利用计算机来生成、处理和显示图形的原理、方法和技术的学科。

计算机图形学的应用

(1) 图形用户接口 (2) 工业应用 (3) 商业领域 (4) 艺术领域 (5) 科学计算可视化+ (6) 虚拟现实 (7) 系统环境模拟

图形和图像的区别

- 表示方法不同: 图形是由基本几何体+ (直线, 点, 圆、曲线、三角形等) 构成的实体, 同时具有几何属性和视觉属性。图像是由很多像素点构成的点阵信息。
- 生成方法不同: 图形是通过计算机算法生成的, 而图像是通过照相机, 摄像机等扫描设备或图像生成软件制作而成。
- 研究侧重点不同: 图形学主要研究如何使用计算机表示几何体, 构建几何模型+、如何通过建立数学模型或者算法把真实的或者想象的物体显示出来。图像处理+主要研究如何将一种图像处理成另一种图像, 包括图像增强、复原、解析和理解、编码、压缩、匹配, 识别等。

第2章 计算机图像处理及显示的基本原理

显卡工作流程

1. 通过数据总线+将要显示的图形或者图像数据送入 GPU (图形处理器+).
2. GPU 对送入的数据进行处理然后送入帧缓冲器+ (或称显存)。
3. 送入帧缓冲器中的数据将被送入视频控制器。视频控制器将根据接口的类型确定处理方式, 完成数模转换+;
4. 视频控制器的输出将送到显示屏。

概念

- 帧缓存/显存: 显存用来存储屏幕上像素的颜色值。帧缓冲器中的单元数目与显示器上的像素数目相同, 单元与像素一一对应, 各单元的数值决定了其对应的像素的颜色。(比如一个像素 24 位)
- 显卡分辨率: 显卡分辨率是指显卡输出给显示器并能在显示器上描绘的像素点的数量。
- 显卡颜色深度: 每个像素包含三种颜色: R, G, B。像素的精细度由组成像素的颜色的位数和照射到像素的颜色的强度决定。显示透明度, 往往还增加 8bits 来表示透明度。

GPU 处理输入的图形数据⁺（渲染流水线）

1. 输入装配阶段。从内存中读入相关的顶点和索引，从而生成几何图形的基本要素（点，线以及三角面）。
2. 顶点着色阶段。完成顶点转换（从局部坐标系⁺转换到齐次裁剪坐标系中），光照（纹理）等各种形式的运算。
3. 曲面细分阶段。将一个大的三角形分解成若干个小的三角形。
4. 几何着色阶段。对输入的点进行筛选，然后输出相应的构成几何图形的基本要素。同时几何着色可以输出一系列的点到内存空间中。
5. 裁剪。将区域之外的物体去除掉，只显示区域之内的物体。
6. 光栅化阶段。将几何图元变为二维图像⁺。第一部分工作：决定窗口坐标中的哪些整型栅格区域被基本图元占用；第二部分工作：分配一个颜色值和一个深度值到各个区域以便进行消隐操作。目的，是找出一个几何单元（比如线段或三角形）所覆盖的像素。
7. 像素着色阶段。通过这些点及相应的属性可以在 GPU 中计算除相应像素的颜色，光照的影响，阴影等处理效果。
8. 输出融合阶段。一些像素无法通过深度测试而被排除，而另一些像素送入，与此前已经写入到后台缓冲区中的像素进行相应的融合处理。

第3章 基本图形生成⁺ ★

画线算法

数值微分画线法（DDA）

- 输入 $P_0(x_0, y_0), P_1(x_1, y_1)$
- 计算增量

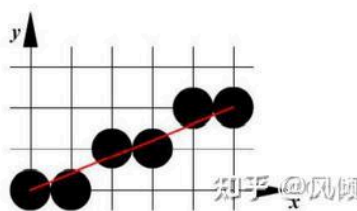
$$\Delta x = \frac{x_1 - x_0}{Length}, \Delta y = \frac{y_1 - y_0}{Length} \text{ 其中 } Length = \max(\text{abs}(y_1 - y_0), \text{abs}(x_1 - x_0))$$
- 循环 $Length$ 次, $x_{i+1} = x_i + \Delta x, y_{i+1} = y_i + \Delta y$, 每步结果对不是整数那个四舍五入

■DDA算法示例：

- ◆两端点坐标分别 $P_0(0, 0)$ 和 $P_1(5, 2)$
- ◆首先计算直线的斜率: $k = (2-0)/(5-0) = 0.4$
 - 因为斜率 k 小于 1, 故采用在 x 方向每次步长为 1, 在 y 方向递增 k 的方法。
- ◆首先绘制初始点 $(0, 0)$, 并确定沿线段路径的其余像素为:

x	y	(int) (y+0.5)
0	0	0
1	0.4	0
2	0.8	1
3	1.2	1
4	1.6	2
5	2	2

img



中点画线法

原理: $d = 2F(x_k + 1, y_k + 0.5) = 2a(x_k + 1) + 2b(y_k + 0.5) + c, d_0 = 2a + b$

- 输入 $P_0(x_0, y_0), P_1(x_1, y_1)$

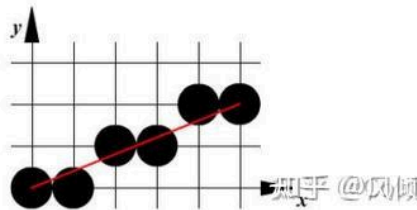
• 当 $a \geq 0$ ，取上点 y 加一 $y_{i+1} = y_i + 1, a+ = \text{delta1}$; ($x++$ 一旦目增)

当 $d < 0$ ，取上点 y 加一 $y_{i+1} = y_i + 1, d+ = \text{delta2}$.

■中点画线算法示例

- 绘制直线：两端点坐标分别 $P_0(0, 0)$ 和 $P_1(5, 2)$ 。
- 解：首先计算直线的斜率 $k=(2-0)/(5-0) = 0.4$ ，因为 $0 \leq k \leq 1$ ，故适用于上述方法。
- 根据公式计算：
- $a = y_0 - y_1 = 0 - 2 = -2$; $b = x_1 - x_0 = 5 - 0 = 5$;
- $d_0 = 2a + b = 1$; $\text{delta1} = 2a = -4$; $\text{delta2} = 2(a+b) = 6$
- 首先绘制初始点 $(0, 0)$ ，并确定沿线段路径其余像素

x	y	d
0	0	1 $=d_0$
1	0	-3 $=1+(-4)$
2	1	3 $=-3+6$
3	1	-1 $=3+(-4)$
4	2	5 $=-1+6$
5	2	1 $=5+(-4)$



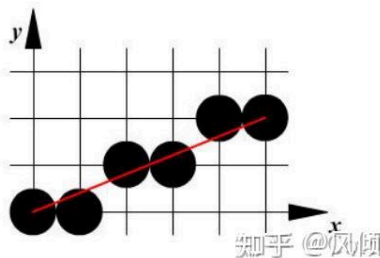
img

Bresenham 画线算法

- 输入 $P_0(x_0, y_0), P_1(x_1, y_1)$
- $e = 2\Delta y - \Delta x$
- 当 $e \geq 0$ ，取上点 $y_{i+1} = y_i + 1, e' = e + 2\Delta y - 2\Delta x$;

当 $e < 0$ ，取下点 $y_{i+1} = y_i, e' = e + 2\Delta y$.

x	y	d
0	0	-1 $=d_0$
1	0	3 $=-1+4$
2	1	-3 $=3+(-6)$
3	1	1 $=-3+4$
4	2	-5 $=1+(-6)$
5	2	-1 $=-5+4$



img

圆形算法

中点画圆算法

基本算法思想就是利用下一个点的绘制要么是右方的点，要么是右下侧的点。

- 初始值 $x = 0, y = R, d = 1 - R$
- $d < 0$ ， y 不变， $d = d + 2x + 3$
- $d \geq 0$ ， y 减一， $d = d + 2x_n - 2y_n + 5$

◆绘制中心在坐标原点，半径R为10，从x=0到x=y的1/8圆弧

◆解：决策变量d的初始值为：

● $d=1-R=-9$

●初始点 (x_0, y_0) 的坐标为 $x=0, y=10$ ，初始增量 $2x+3=3$ ，则后续的决策变量与像素

i	x	y	d _i	$2x+3$	$2(x-y)+5$
0	0	10	-9 = d_0	3	
1	1	10	-6 = $-9+3$	5	
2	2	10	-1 = $-6+5$	7	
3	3	10	6 = $-1+7$		-9
4	4	9	-3 = $6+(-9)$	11	
5	5	9	8 = $-3+11$		-3
6	6	8	5 = $8+(-3)$		1
7	7	7	6 = $5+1$		

img

Bresenham画圆算法

利用下一个点的绘制要么是右方的点，要么是右下方的点，要么是下方的点 不断判定三个距离哪个最短则选择哪个。

- 初始化 $x=0, y=R, d=2(1-R)$
- $d < 0, 2(d+y)-1 \leq 0$: $x=x+1, d=d+2x+3$
- $d < 0, 2(d+y)-1 > 0$: $x=x+1, y=y-1, d=d+2(x-y+3)$
- $d = 0$: $x=x+1, y=y-1, d=d+2(x-y+3)$
- $d > 0, 2(d-x)-1 \leq 0$: $x=x+1, y=y-1, d=d+2(x-y+3)$
- $d > 0, 2(d-x)-1 > 0$: $y=y-1, d=d-2y+3$

■Bresenham画圆算法示例

◆绘制中心在坐标原点，半径R为10，从x=0到x=y的1/8圆弧

●决策变量d的初始值为： $d=2(1-R)=-18$

●初始点 (x_0, y_0) 的坐标为 $x=0, y=10$ ，初始增量 $2x+3=3$ ，则后续的决策变量与像素

i	x	y	d	$2(d+y)-1$	$2x+3$	$2(x-y)+3$	$2(d-x)-1$	$-2y+3$
0	0	10	-18	-17	3			
1	1	10	-15	-11	5			
2	2	10	-10	-1	7			
3	3	10	-3	13		-8		
4	4	9	-11	-5	11			
5	5	9	0			-2		
6	6	8	-2	11		2		
7	7	7	0			6		

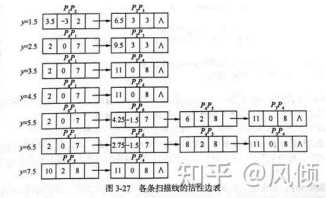
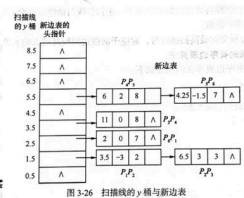
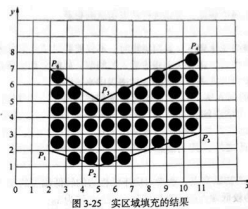
img

线宽和线型的处理

- 线宽处理：刷子绘制法，方形刷子绘制法，实区域填充法产生线宽
- 线型处理：看成一个由布尔值对应的像素。每种线型对应一定的长度的像素，每个像素对应相应的布尔值。如果该像素对应的布尔值为1，则绘制该像素。如果对应的布尔值为0，则不绘制该像素。

· 位置表示法：在正负半轴上表示多边形区域

- 转角法：计算点与多边形各个顶点的连线所构成的夹角之和（带方向），为0表明是在区域外；如果为360°表明在区域内。
- 射线法：水平（右）或者垂直（下）方向做射线。判断该射线与多边形交点的个数。交点数为偶数在图形外；交点数为奇数在图形内。
- 扫描线法：
 - 求交、排序、两两配对、线段着色
 - 注意顶点求交点规则：把所有极值顶点当成两个点
 - （活性边表法好像不考？）边表的结构为 $[x, y_{max}, \frac{1}{k}]$ 。x: 当前扫描线与活性边的交点， y_{max} : 活性边较高端点的y坐标值， $\frac{1}{k}$: 活性边所在直线斜率的倒数。



活性边表

- 边填充法：该多边形每一条边右边区域像素的颜色全部取补
- 栅栏填充法：该多边形每一条边与栅栏之间像素的颜色全部取补
- 种子填充法：四连通/八联通，从某个像素点开始BFS/DFS整个像素点开始个区域（缺点：堆栈大量使用）
- 扫描线种子填充法：
 - 栈顶像素出栈。
 - 沿扫描线对出栈像素的左右像素进行填充，直到遇到边界像素xl和xr为止
 - 在区间[xl,xr]中检查与当前扫描线相邻的上下两条扫描线，则把每个非边界、未填充的像素区间的最右像素取为种子像素入栈。

图形反走样技术

问题：阶梯状边界；图形细节失真；狭小图形遗失；动画序列中时隐时现，产生闪烁。本质是用离散的像素去表示连续图形，从而引起失真。

Bresenham 区域反走样技术：将多边形每条边与具有一定面积的像素相交的面积大小分为8个等级。相交面积 0-1/8 1/8-2/8.....

$$S = (y_{i+1} + y_i)/2 = (y_i + y_i + m)/2 = y_i + m/2 = e + m/2$$

曲线生成

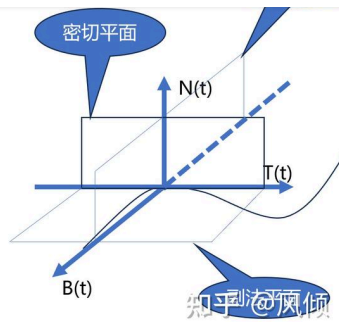
要求：

- 唯一性
- 几何不变性：用相同的方法去拟合平面空间中不同坐标系下相同的几个点，得到的拟合曲线不变。直角坐标系不具备几何不变性。
- 易于定界：矢量坐标系中 $p(t) = [x(t), y(t), z(t)]$ ，由于每一个变量都用统一的t标量来描述，非常容易确定其边界。
- 统一性：可以在不增加其他参数的情况下进行维数的扩充。 $P(t) = [x(t) \ y(t)] \Rightarrow P(t) = [x(t) \ y(t) \ z(t)]$

参数样条曲线

- 零阶几何连续性： G^0 连续性，首尾相接
- 一阶几何连续性： G^1 连续性，首尾相接，且切矢量方向连续（一阶导数成比例）
- 一阶几何连续性： G^2 连续性，首尾相接，一二阶导数成比例，曲率连续。

$T(t)$ 为曲线 $P(t)$ 在 $t = t_0$ 的**切矢量**， $N(t)$ 为**主法矢量**，其与 $T(t)$ 垂直。副法矢量 $B(t)$ 与切矢量 $T(t)$ 构成的平面称为**副法平面**，副法矢量 $B(t)$ 与主法矢量 $N(t)$ 构成的平面称为**法平面**；主法矢量 $N(t)$ 和切矢量 $T(t)$ 所构成的平面称为**密切平面**。



参数样条曲线数学表示：

$$P(t) = \begin{bmatrix} x(t) \\ y(t) \\ z(t) \end{bmatrix} = \begin{bmatrix} x_0 & x_1 & \cdots & x_n \\ y_0 & y_1 & \cdots & y_n \\ z_0 & z_1 & \cdots & z_n \end{bmatrix} \begin{bmatrix} 1 \\ \vdots \\ t^n \end{bmatrix} = C \cdot T \quad t \in [0, 1]$$

Hermite 插值样条：已知每个型值点的**位置矢量**及其切矢量，然后绘制出的样条曲线。

Bezier 曲线的生成

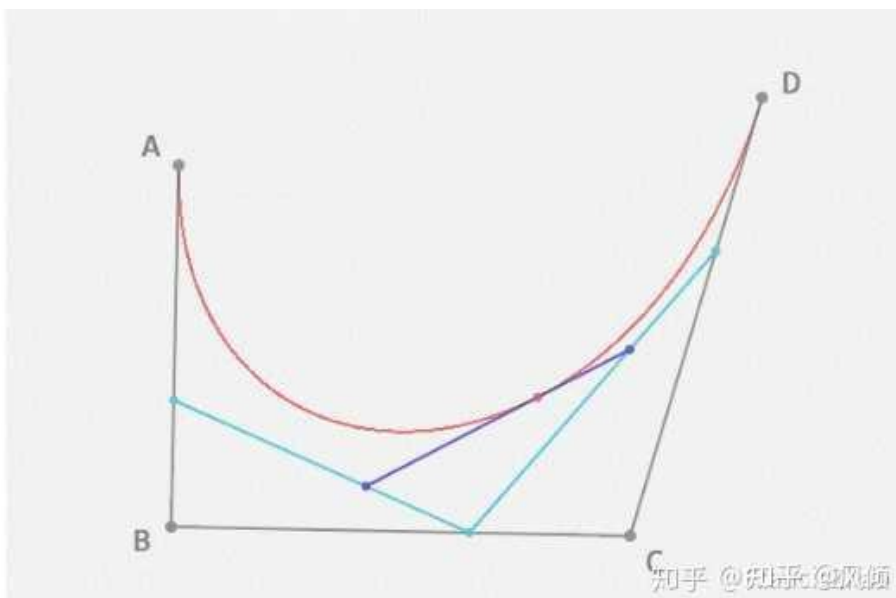
Bezier 曲线通过**控制点**来控制曲线的形状。把相邻的点连接起来形成一个特征多边形，并将其作为曲线的轮廓线，然后在每个特征多边形顶点配以**伯恩斯坦** (Bernstein) 多项式作为权函数，对特征多边形的各顶点进行加权求和。

$$P(t) = \sum_{i=0}^n P_i \cdot BEZ_{i,n}(t) \quad t \in [0, 1]$$

$$BEZ_{i,n}(t) = C_n^i t^i (1-t)^{n-i}, \quad t \in [0, 1]$$

De Casteljau 算法：递归法绘制点

- (1) 用直线连接所有相邻的 n 个控制点，得到其特征多边形。
- (2) 如果 $t \in [0, 1]$ ，则在特征多边形的每一条边上按照 $t:(1-t)$ 的比例寻找一组新的点，构成新的多边形，该多边形的边比前面的多边形少一条边。
- (3) 在新的多边形上，重复 (2) 的操作得到一组新的多边形。在 n 次这种操作之后，将得到一个点，该点即为曲线上的点。



B样条曲线的生成

B样条曲线由 $n+m+1$ 个控制点构成，其中 $P_0 P_n$ 这个 $n+1$ 个控制点控制其第一段曲线， $P_1 P_{n+1}$ 这个 $n+1$ 个控制点控制其第二段曲线，..., 依次类推，总共 $m+1$ 段曲线构成B样条曲线。

$$P(t) = \sum_{i=0}^n P_i \cdot B_{i,n}(t) \quad t \in [0, 1]$$

$$B_{i,n}(t) = \frac{1}{n!} \sum_{j=0}^{n-i} (-1)^j C_{n+1}^j (t + n - k - j)^n$$

其第 i 段曲线可表示为： $P_{i,n}(t) = \sum_{k=0}^n P_{i+k} \cdot B_{k,n}(t) \quad t \in [0, 1]$

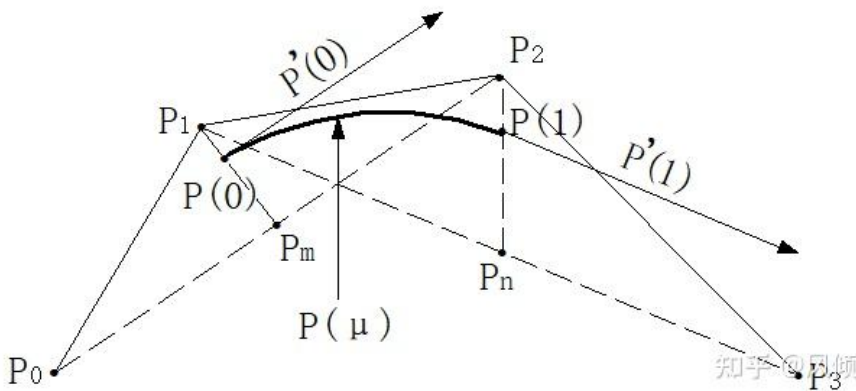


image-20231230132401533

第4章 图形变换与裁剪 ★

图形矩阵变换+

注：课程默认使用行向量表达，复合变换矩阵+要写在列向量的右侧（从左向右结合）

$$T_{3D} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ b_x & b_y & b_z & 1 \end{bmatrix}, S_{3D} = \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$\mathbf{R}_x = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & \sin \theta & 0 \\ 0 & -\sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}_y = \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \mathbf{R}_z$$

$$= \begin{bmatrix} \cos \theta & \sin \theta & 0 & 0 \\ -\sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

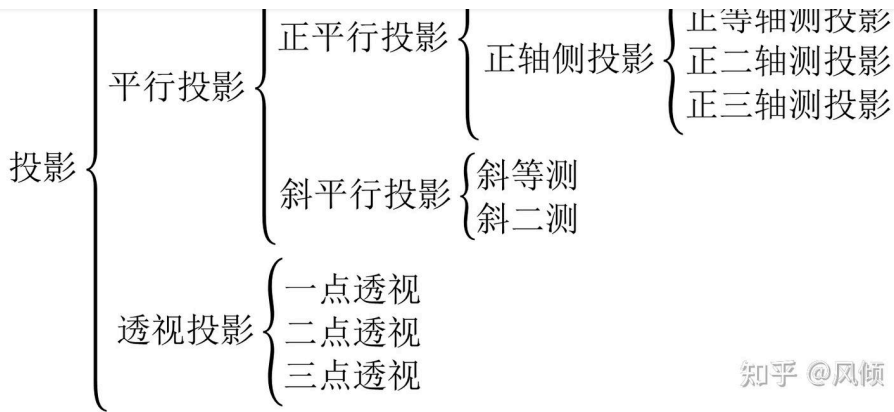
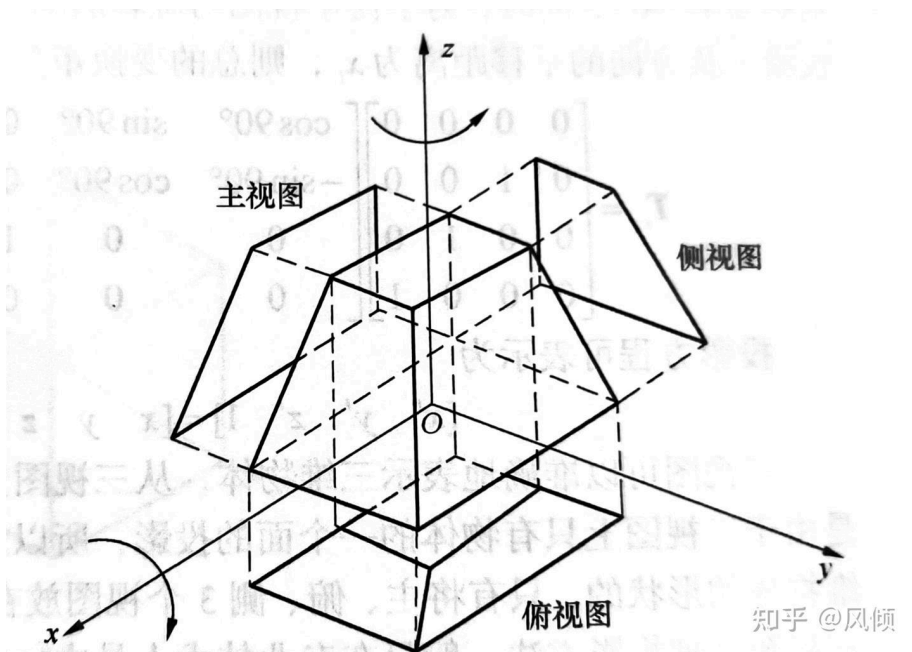


image-20231230135219739

平行投影



投影平面为 xOz 面

主视图: y 拍扁

$$T_v = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

俯视图+: z 拍扁, 绕 x 轴顺时针旋转 90° , 然后沿 z 方向平移一段距离。

$$T_h = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos(-90^\circ) & \sin(-90^\circ) & 0 \\ 0 & -\sin(-90^\circ) & \cos(-90^\circ) & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -z_p & 1 \end{bmatrix}$$

侧视图: x 拍扁, 绕 z 轴逆时针旋转 90° 到 xOz 面, 然后在 x 轴方向上移动一定距离。

$$\begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 0 & 1 & 0 \\ -x_L & 0 & 0 & 1 \end{bmatrix}$$

正轴测投影：将物体绕Z轴逆时针旋转某个角度（比如 β 角），然后再绕X轴顺时针旋转 α 角，然后再向xOz面做正平行投影。

$$R_{zx} = R_z R_x$$

$$= \begin{bmatrix} \cos \beta & \sin \beta & 0 & 0 \\ -\sin \beta & \cos \beta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \alpha & \sin \alpha & 0 \\ 0 & -\sin \alpha & \cos \alpha & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

透视投影+

1. 一点透视：也称为平行透视。即投影平面与投影对象所在坐标系的一个坐标平面平行。
2. 两点透视：也称为成角透视。即投影平面与投影对象所在坐标系的一个坐标轴平行，而与另两个坐标轴成一定角度。
3. 三点透视：也称为斜透视。即投影平面与投影对象所在的坐标系的坐标轴均不平行。

设投影中心E在Z轴上距离原点O点d的位置上，投影平面垂直于z轴，并位于投影中心和投影对象之间。则有：

$$\frac{x'}{d} = \frac{x}{-z+d} \quad \frac{y'}{d} = \frac{y}{-z+d} \quad x' = \frac{x}{(-\frac{z}{d})+1}$$

$$y' = \frac{y}{(-\frac{z}{d})+1} \quad r = -1/d, \text{ 则:}$$

$$T = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & r \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

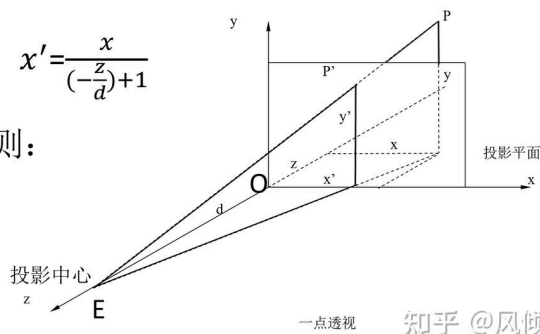


image-20231230135755972

灭点：不平行于投影平面的平行线（如平行于z轴的平行线）的投影的汇聚点，这个点称为灭点。（一定落在同一个面上）

主灭点：平行于三维坐标系坐标轴的平行线在投影平面上形成的灭点。

$[x', y', z', 1] = [0, 0, 1/r, 1]$ 其对应的投影约矩阵为：

$$[x' \ y' \ z' \ H] = [x \ y \ z \ 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} = [x \ y \ z \ qy + 1]$$

如果灭点在X轴 $[1/p, 0, 0, 1]$ 和Y轴 $[0, 1/q, 0, 1]$ 上，则对应的投影矩阵分别为：

$$P_x = \begin{bmatrix} 1 & 0 & 0 & p \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, P_y = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & q \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

图形裁剪

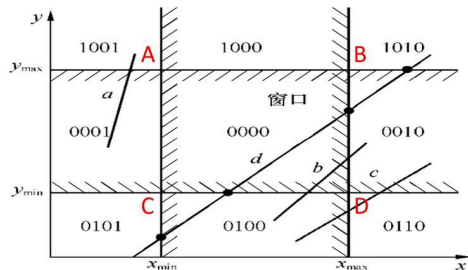
1. 对线段端点编码，定义为它所在区域的编码，

2. 快速判断“完全可见”，若两端点编码均为0，则完全可见。 $RC0=0$ 且 $RC1=0$

3. 快速判断“完全不可见”，线段两端点编码的逻辑位“与”运算结果非零，则完全不可见。

$RC0 \& RC1 \neq 0$ 说明直线段位于窗外的同一侧

4. 若2)、3)不满足，逐个端点判断其编码 $CtCbCrCl(C3C2C1C0)$ 中位是否为“1”，若是，则需求交，交点的x坐标进行排序，从而将原始线段分割成最多五段线段（四个交点）。然后根据（1）（2）步判断这些线段是否在区域内。



$$x_{min} = xa \quad x_{max} = xb$$

$$y_{min} = yc \quad y_{max} = ya$$

知乎 @风倾

编码

中点分割裁剪算法

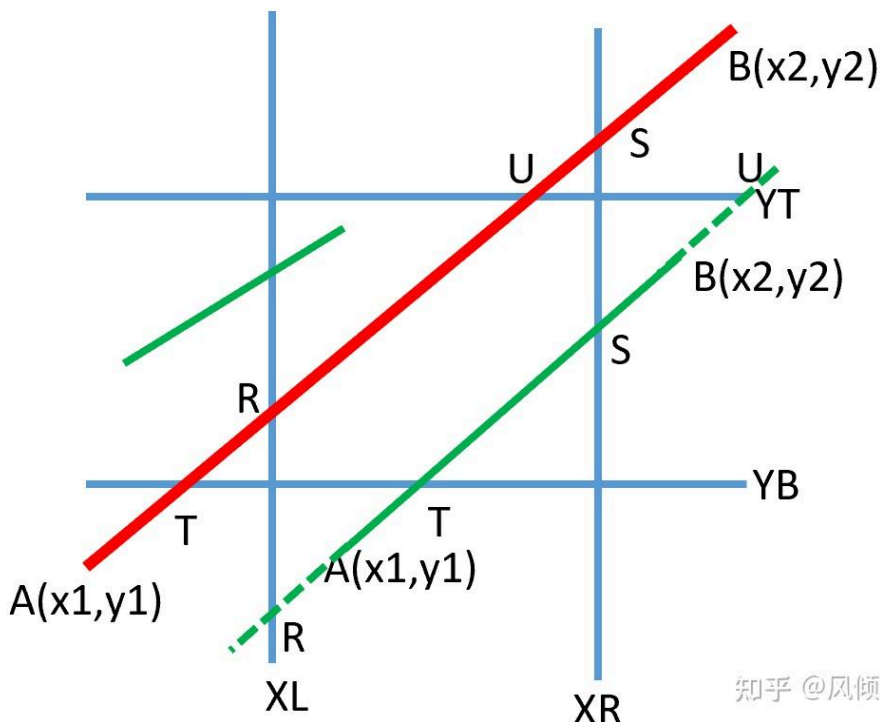
中点分割法的思想是通过不断求线段的中点，并采用 Cohen-Sutherland 算法可见性判断，然后确定可见区域内的线段。

固定 P_1 ，测试 P_2 是否在窗口内，若是，则 P_2 是离 P_1 点最远的可见点。否则，将线段 P_1P_2 对分，求出中点 P_m ，编码判断线段 P_mP_2 是否全部在窗口外，若是，则舍弃 P_mP_2 ，用 P_1P_m 代替 P_1P_2 ；若不是，则用 P_mP_2 代替 P_1P_2 。

本质就是固定一个端点求，用二分法+求这个端点的最远的可见点。

Liang（梁友栋）-Barsky裁剪算法

无论何种情况下，在裁剪区域之内的线段均为： $AB \cap RS \cap TU$



知乎 @风倾

记 $L = \max[x_{min}, \min(x_A, x_B)]$, $R = \min[x_{max}, \max(x_A, x_B)]$ ，不等式+若有不成立，则不存在可见线段，不等式为：

$$\{ \min(x_T, x_U) \leq K$$

$$\{ x_T \leq K$$

$$\{ x_U \leq K$$

A B 斜率大于 0 时，可见线段的端点坐标：

$$x_\alpha = \max(L, x_T)$$

$$x_\beta = \min(R, x_U)$$

A B 斜率小于 0 时，可见线段的端点坐标：

$$x_\alpha = \max(L, x_U)$$

$$x_\beta = \min(R, x_T)$$

多边形裁剪

Sutherland-Hodgman 算法

基本思想：用裁剪窗口的每一条边去裁剪某个多边形

1. 边 $P_i P_{i+1}$ 可见一侧，输出 P_{i+1} 。
2. 边 $P_i P_{i+1}$ 不可见一侧，此时没有新的顶点输出。
3. 边 $P_i P_{i+1}$ 离开窗口，输出交点。
4. 边 $P_i P_{i+1}$ 进入窗口，该交点和顶点 P_{i+1} 都输出。

问题：裁剪凹多边形可能会出现多余的边

Weiler-Atherton 算法

可适用于任意类型的多边形，包括有空洞的。将裁剪多边形称为 CP，被裁剪多边形称为 SP。

1. 算法首先将 CP 和 SP 的顶点按照外部边界取顺时针，内部边界取逆时针的方式将其构成环形链表。
2. 求出 SP 与 CP 边线的所有交点，并将这些交点插入到 SP 和 CP 的环形链表中，并标明交点是出点还是进点。
3. 算法从任意一个进点开始沿着 SP 的边线按照边线所标示的方向搜集顶点序列，当遇到出点时，则沿着 CP 的边线所标示的方向搜集顶点序列，当遇到进点时，则沿着 SP 的边线所标示的方向搜集顶点序列。

搜索所有点。

图形消隐

背面剔除算法

在取景变换（将物体从世界坐标系转换到摄像机坐标系）之前，将那些背离视点方向而不可见的景物表面剔除。

1. 计算多边形的法向量 N 和视线向量 V。
2. 计算法向量 N 和视线向量 V 的夹角 θ 的余弦 $\cos\theta = N \cdot V$ 。
3. $\cos\theta < 0$ ，即 $\theta > 90^\circ$ ，则该多边形表面为背面，是不可见的

画家算法

将多边形按照离视点的远近进行排序（P221 判断遮挡，分割），然后先画出被遮挡的多边形，再画出不被遮挡的多边形，用后画的多边形覆盖先画的多边形，从而达到自动消隐的目的。

Weiler-Atherton 算法

算法思想：采用裁剪算法的思想对隐藏面进行消隐。

1. 先进行初步的体及投影序，即对又投影到屏幕上的景物表面多边形按目视点的z值进行排序，形成景物多边形表。
2. 以当前具有最大z值（即离视点最近）的景物表面作为裁剪多边形CP。
3. 用CP对景物多边形表中排在后面的景物表面进行裁剪，产生内部多边形Pin和外部多边形Pout。
4. 由于多边形顶点离视点最近的多边形表面不一定是真正排在最前面的可见面，因此，需比较CP与内部多边形Pin的深度，检查CP是否是真正离视点较近的多边形。如果是，则CP为可见表面，而位于裁剪多边形CP之内的多边形Pin为当前视点的隐藏面，可以消去该隐藏面；如果不是，则选择Pin为新的裁剪多边形，重复步骤3。
5. 将位于裁剪多边形之外的景物表面Pout组成外裁剪结果多边形表，取表中深度最大即排在最前面的表面为裁剪多边形，重复步骤3，继续对表中其他景物表面进行裁剪。
6. 上述过程递归进行，直到外裁剪结果多边形表为空时为止

BSP树算法

算法思想：平面分割**二叉树**+

先在场景选取一剖分平面P1（与视点垂直的平面），将场景空间分割成两个**半空间**。它相应地把场景中的景物分成两组，相对于视点而言，一组景物多边形位于P1的前面，作为BSP树的左孩子（front），另一组景物多边形位于P1的后面，作为BSP树的右孩子（Back），如果有物体与P1相交，就把它分割为两个物体分别标识为A和B，再用平面P2对所生成的两个子空间继续进行分割，并对每一子空间所含景物进行分类。上述空间剖分和景物分类过程递归进行，直至每一子空间中所含景物少于给定的阈值为止。

优先绘制标识为“back”的子空间中所含的景物，这样可使得前面的物体覆盖后面的物体，从而实现物体的消隐。

深度缓冲区算法

目前大多数的硬件实现方法

算法思想：对投影到显示屏上的每一个像素所对应的多边形表面的深度进行比较，然后取最近表面的属性值作为该像素的属性值。**Z缓冲器（Z-buffer，深度缓冲区）**，深度缓冲器只是帧缓冲器的扩充。

1. 将场景中的所有多边形通过取景变换、透视变换变换到屏幕坐标系中，物体的深度比较可通过它们z值的比较来实现。
2. 初始化深度缓冲存储器和帧缓冲器。深度缓冲器应初始化为离视点最远的最大z值，帧缓冲器应初始化为背景的属性值。
3. 扫描待显示的每一个多边形，比较当前多边形所覆盖的每一个像素点的深度值，如果其深度值比深度缓冲区中当前像素的深度值小，则取代原来的深度值，并将对应像素的属性值保存到帧缓冲区对应的位置中。反之，则不做任何处理。循环该步骤，直到处理完所有的多边形。

Warnock算法

该算法将整个观察范围细分成越来越小的矩形单元，直至每个单元仅包含单个可见面片的投影或不包含任何面片。

单纯窗口：景物已经足够简单，比如没有任何可见物体，或者窗口已被一个可见面片完全充满

算法思想：将非单纯的窗口四等分为4个子窗口（**四叉树**），对每个子窗口再进一步判别是否是单纯的，直到窗口单纯或窗口边长已缩减至一个像素点为止。

1. 对每个窗口进行判断，若画面中所有多边形均与此窗口分离，即此窗口为空，则按背景光强或颜色直接显示而无需继续分割。

3. 若窗口与一个多边形相交，则窗口内多边形外的区域按背景光强或颜色填充，相交多边形内位于窗口内的部分按多边形相应的光强或颜色填充。
4. 若窗口被一个多边形包围且窗口内无其他的多边形，则窗口按此包围多边形相应的光强或颜色填充。
5. 若窗口至少被一个多边形包围且此多边形距离视点最近，则窗口按此离视点最近的包围多边形的相应光强或颜色填充。
6. 若以上条件都不满足，则继续细分窗口，并重复以上测试。

四元素，看不懂...

第5章 模型生成方法

模型的表示方法

三角面：顶点+索引

参数多项式曲面

$$\begin{cases} x = x(u, v) \\ y = y(u, v) \\ z = z(u, v) \end{cases} \quad (u, v) \in [0, 1] \times [0, 1]$$

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n a_{ij} u^i v^j = U^T A V \quad (u, v) \in [0, 1] \times [0, 1]$$

Bezier 曲面

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} BEZ_{i,m}(u) BEZ_{j,n}(v) \quad u \in [0, 1], v \in [0, 1]$$

B 样条曲面

$$P(u, v) = \sum_{i=0}^m \sum_{j=0}^n P_{ij} B_{i,k}(u) B_{j,h}(v) \quad u \in [0, 1], v \in [0, 1]$$

3D 地形模拟

Diamond-Square 算法

1. 给定一个四边形,四个顶点的坐标(包含高度), 定义一初始高度因子h和一缩放倍数b。
2. 迭代, 先求出四边形的中心点高度为顶点平均高度+h*r的随机数, 再分别求出4个边的中点, 高度为两端点高度的平均值+h*r
3. 修正高度因子 h=h/b, 用(2)的方法分别对这4个小正方形进行处理。直到达到所期望的山形细腻程度。

植物形态模拟

L系统的植物形态模拟

- F: 从当前位置向前走h的长度, 同时画线
- G: 从当前位置向前走h的长度, 但不画线
- +: 从当前方向向左转δ角度

定义一个产生式，通过产生式生成新的字符串。如果进行多次迭代，可生成更长的新的字符串，然后根据所产生的字符串来生成对应的树。

根据前面的符号定义规则，字符串 $F[+F]-F[-F]+F$ 所生成的树如右上图所示。

经过一次迭代之后的字符串为 $F[+F]-F[-F]+F[+F[+F]-F[-F]+F]-F[+F]-F[-F]+F[-F[+F]-F[-F]+F]+F[+F]-F[-F]+F$ ，对应的树形结构如如下图所示。

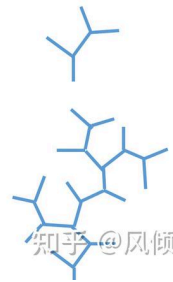


image-20231231094949109

迭代函数算法 (Iterated Function System, IFS)

IFS 系统由一个压缩仿射变换集 $X = w_1, w_2, \dots, w_N$ 和对应概率集 $P = p_1, p_2, \dots, p_N$ 组成

$$\sum_{j=1}^N p_j = 1, p_j > 0, j = 1, 2, \dots, N$$

$$w_j \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} a_j & b_j \\ c_j & d_j \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} + \begin{pmatrix} e_j \\ f_j \end{pmatrix}, j = 1, 2, \dots, N$$

输入：初始点的坐标 x, y ; 输出：屏幕上显示的树

1. 输入 N 或者设定 N 为某个固定值，输入或设置迭代次数 M 。
2. 随机生成 N 个 W 概率值，并保存到 $w[N]$ 。
3. 随机生成 N 个概率值，并保存到 $P[N]$ 。
4. 随机生成概率值 p 。
5. 轮盘选择法。如果 $P < P[0]$, 则 $w = w[0]$; 反之，如果 $P < p[0] + P[1]$, 则 $w = w[0] + w[1]$; ...
6. 根据前面的公式更新 x, y 。
7. 在屏幕上 (x, y) 的位置以一定的亮度或者颜色显示该像素。
8. 循环执行步骤 (4) ~ (7) M 次。

粒子系统⁺：雨雪现象模拟、液态流体模拟⁺

1. 生成一定数量的初始粒子，对其各个属性进行赋值。
2. 更新粒子的运动参数⁺，比如速度，加速度，位置，生命周期等。
3. 针对每一个粒子，在粒子对应的位置上绘制位图，并进行位移的映射，背景融合等。
4. 删除已经消亡的粒子，释放其占用的资源。

在模拟时都是在每一帧对其属性进行更新。

- 基于粒子系统的喷泉模拟
- 基于粒子系统的瀑布模拟

气态流体模拟

细胞自动机⁺：又称为元胞自动机 (Cellular Automata)，是一种在空间和时间上都离散的演化动力系统⁺。每个元胞之上都有若干种离散或者连续的状态，这些状态在每一个时间步都会按照相同的规则发生变化，于是形成了整个元胞自动机的演化过程。

例子：火焰的模拟， $s_{i,j}^t$ 局部温度 $u_{i,j}^t$ 燃料供应⁺.....

光照模型

- 光照模型分类：几何光照模型（局部光照模型和整体光照模型）、物理光照模型
- 光源的分类：点光源、线光源、面光源⁺、体光源

光照基本模型

漫反射⁺光(diffuse Reflection)：当光线照射到物体表面，有一部分光线将进入物体内部，部分会被吸收，而另一部分会从内部向各个方向散射并返回表面，这就形成漫反射光。

$$c_d = \max(\mathbf{L} \cdot \mathbf{n}, 0) \cdot B_L \otimes m_d$$

环境光照 (Ambient Light)：从其他物体发射过来的光线照射到物体所形成的间接光照。

$$c_a = A_L \otimes m_d$$

镜面光照 (Specular reflection)：一部分光将被反射，另一部分光被折，被反射的部分称为镜面反射光，只有观察者在特定的角度才能看到。

$$c_s = \max(\mathbf{L} \cdot \mathbf{n}, 0) \cdot B_L \otimes R_F(\alpha_h) \frac{m+8}{8} (\mathbf{n} \cdot \mathbf{h})^m$$

光照整体模型：

$$\text{LitColor} = c_a + c_d + c_s$$

$$= A_L \otimes m_d + \max(\mathbf{L} \cdot \mathbf{n}, 0) \cdot B_L \otimes \left(m_d + R_F(\alpha_h) \frac{m+8}{8} (\mathbf{n} \cdot \mathbf{h})^m \right)$$

- L: 光源的光向量 n: 表面法向 h: 光向量与观察向量之间的中间向量 A_L : 入射的环境光量。
 m_d : 根据表面漫反射率而反射的入射光量。 $\mathbf{L} \cdot \mathbf{n}$: 光线与法向量之间夹角的余弦。 α_h : 中间向量 h 与光向量之间的夹角。 $R_F(\alpha_h)$: 中间向量 h (由表面点指向观察点的单位向量⁺) 所反射到观察者眼中的光量。 m: 控制表面的粗糙度。 $(\mathbf{n} \cdot \mathbf{h})^m$ 线 h 与宏观表面法向 n 之间夹角为 θ_h 的所有为平面片段。 $\frac{m+8}{8}$: 在镜面发射过程中，为模拟能量守恒所采用的归一化因子。

环境光照

- 环境光模型（物体表面对环境光反射的强度）
- 漫反射模型（观察者从不同角度观察到的反射光具有同样的亮度，这样的反射光成为漫反射光）
- 镜面反射（反射光 => 对入射光的直接反射）
- Phong 模型（环境光 + 漫反射光 + 镜面反射光）

整体光照模型：光线追踪⁺

明暗处理

Gouraud 明暗处理

基本思想：对离散的顶点颜色采样进行双线性插值⁺得到内部点颜色。

- 计算每个多边形顶点的平均单位法矢量
- 对每个顶点根据简单光照模型计算光强
- 在多边形表面上对顶点颜色进行线性插值

特点：（1）只适用于简单的漫反射模型（2）线性光强度插值会引起马赫带效应⁺

Phong 明暗处理

- 计算多边形顶点处曲面法向矢量的平均值。

纹理细节模拟

颜色纹理：将图片映射到物体表面的方式来实现。将图片的四个角点的坐标分别定义为(0,0) (0,1) (1,1) (1,0)。三角形每一个顶点均对应图片的一个(u,v)坐标。

几何纹理：物体表面的微观几何形状（粗糙程度）。物体表面法向量矢量的修改可通过在各采样点*的位置上增加一个扰动函数，对其做微小的扰动，从而改变表面的微观几何现状来实现。

过程纹理：通过过程迭代函数生成纹理。目的是用简单的参数来逼真的描述复杂的自然纹理细节。例如：噪声函数，湍流函数等来动态的生成天空，水流等。

编辑于 2024-01-07 17:29 · 四川

计算机图形学 计算机图形学 计算机 计算机图形学和可视化

千元内高性价比蔡司镜片推荐

预算有限又想上蔡司，佳锐真的可以。上手比较直观就是亮，看东西高清透亮。还防紫外线，日常要的高清、舒适、防护都给足，很全能 [查看详情](#)

蔡司 的广告

赞同 86 3 条评论 238 33 分享 申请转载



未登录用户

3 条评论

默认 最新



月球

膜拜翕神

2025-11-13 · 四川

回复 喜欢



邢台白博美犬舍

请问一下，使用的是哪本教材？感谢。

2024-01-01 · 河北

回复 喜欢



风倾 作者

苏小红的

2024-01-02 · 四川

回复 1

推荐阅读

计算机图形学

一,单选题 1.枚举出图形中所有点的表示方法是() A. 图形 B. 图像 C. 参数法 D. 点阵法 [答案]:D 2. 下面哪个设备不是计算机图形学的输入设备 A. 光笔 B. 键盘 C. 扫描仪 D. 显示器 [答案]:D 3. 下面哪...

小医僧

计算机图形学复习大纲总结

图形学基本概念 1. 什么是计算机图形学？研究通过计算机，将数据转化为图形，并在专门显示设备上显示的原理、方法和技术的学科。也就是如何用计算机生成、处理和显示图形的一门学科。 2. 计...

风倾

发表于课程复习

计算机图形学笔记(六)：现代图形学前沿

计算机图形学笔记(六)：现代图形学前沿主要是一些前沿技术 目录实时渲染优化技术 - LOD、遮挡剔除与GPU优化基于物理的渲染(PBR) - 微表面理论与材质建模体积渲染与参与介质 - 云雾烟尘的渲...

crayon

再学计算机图形学入门

在网上查资料时，无意间发现了一门课叫《现代计算机图形学入门》。于是事隔将近3年后，我再一次尝试图形学入门。这次学习从8月20号开始，一直到10月11日，约持续了一个半月。但是这次的学...

重归混沌

发表于重归混沌的 ...